

CloudMart – Deployment & Operations Runbook

AWS Serverless E-Commerce Application

Purpose: A practical guide for deploying, validating, monitoring, troubleshooting, and recovering the CloudMart application on AWS.

1. Document Overview

CloudMart is a cloud-native, serverless e-commerce application deployed on AWS. The project uses AWS CloudFormation/SAM for infrastructure, GitHub Actions with GitHub OIDC for CI/CD, Lambda and API Gateway for backend APIs, RDS MySQL for application data, S3 for reports/artifacts, EventBridge for scheduled/event-driven processing, CloudWatch for monitoring, and SNS for notifications. A Flask dashboard is hosted on EC2.

2. Repository Structure

- .github/workflows/deploy.yaml – CI/CD deployment workflow
- cloudformation/ – infrastructure templates
- cloudformation/network-stack.yaml – VPC/networking
- cloudformation/data-stack.yaml – data/storage resources
- cloudformation/iam-stack.yaml – IAM roles and permissions
- cloudformation/auth-stack.yaml – authentication/authorizer resources
- cloudformation/product-stack.yaml – product APIs/resources
- cloudformation/orders-stack.yaml – order APIs/resources
- cloudformation/monitor-stack.yaml – monitoring, metrics, alarms/dashboard
- lambdas/ – authorizer, product, order, report, and schema initialization functions
- dashboard/ – Flask dashboard
- layer/ – shared Lambda dependencies
- parameters/parameters.json – deployment/environment configuration
- Documents/ – project documentation

3. AWS Architecture – High Level

Developer → GitHub → GitHub Actions → GitHub OIDC → IAM deployment role → STS temporary credentials → CloudFormation/SAM → AWS infrastructure.

Client → API Gateway → Lambda Authorizer → Product/Order Lambda → RDS MySQL.

EventBridge → Report Lambda → S3 reports. CloudWatch collects logs/metrics and triggers alarms; SNS sends notifications. EC2 hosts the Flask dashboard.

4. Deployment Prerequisites

- AWS account with permission to deploy the CloudMart infrastructure.
- GitHub repository containing the CloudMart source code.
- AWS OIDC provider for token.actions.githubusercontent.com configured for GitHub Actions.
- IAM deployment role: cloudmart_githubaction_role (or the role configured in your repository).
- GitHub Actions secret ARN_ROLE containing the deployment role ARN.
- Required SSM Parameter Store configuration for the application.
- Database password/configuration prepared according to the project's deployment configuration.

- AWS SAM CLI and AWS CLI available locally when local validation or troubleshooting is required.
- Correct AWS Region configured in the project parameters.

5. Important Security Rules

- Never commit database passwords, access keys, tokens, or other secrets to GitHub.
- Use GitHub OIDC and temporary AWS credentials instead of long-lived AWS access keys in GitHub Actions.
- Keep application secrets/configuration in the configured AWS parameter/secret mechanism.
- Do not place sensitive tokens directly in README files or screenshots.
- Use least-privilege IAM permissions wherever possible.

6. CI/CD Deployment Procedure

6.1 Push Code

Commit and push the required changes to the deployment branch configured by the workflow (for the standard flow, main).

```
git status

git add .

git commit -m "Deploy CloudMart updates"

git push origin main
```

6.2 GitHub Actions Flow

1. Workflow starts after the configured GitHub event.
2. Repository code is checked out.
3. CloudFormation/SAM templates are validated/linted as configured.
4. GitHub requests an OIDC token.
5. AWS trusts the GitHub OIDC provider and allows the configured IAM role to be assumed.
6. STS provides temporary AWS credentials to the workflow.
7. Infrastructure/application stacks are deployed in dependency order.
8. Schema initialization creates/updates the required database schema.
9. The workflow reports success or failure.

7. Expected Stack Deployment Order

#	Stack	Purpose
1	Network	VPC, subnets, route tables, security groups and related networking
2	Data	Database/storage resources required by the application
3	IAM	Application Lambda roles and permissions
4	Auth	Lambda Authorizer and authentication-related API configuration
5	Product	Product/inventory API resources
6	Order	Order API resources and related processing
7	Monitor	CloudWatch metrics, alarms, dashboard and monitoring

		resources
--	--	-----------

8. Post-Deployment Validation

8.1 Infrastructure

- Confirm all expected CloudFormation stacks are CREATE_COMPLETE or UPDATE_COMPLETE.
- Confirm Lambda functions exist and have the expected runtime/configuration.
- Confirm API Gateway API/stages are available.
- Confirm RDS is available and reachable from the Lambda VPC configuration.
- Confirm S3 report/deployment buckets exist as configured.
- Confirm CloudWatch dashboard and alarms exist.

8.2 Authentication Test

Test requests with a valid authorization token and verify that the Lambda Authorizer allows the request.

Test with an invalid or missing token and verify that the protected API returns an authorization failure.

Verify role-based access: Admin, Product, and Customer permissions should match the application implementation.

8.3 Product API Test

- GET /products – list products.
- GET /products/{id} – retrieve a product.
- POST /products – create a product when the caller has permission.
- PATCH /products/{id} – update a product when permitted.
- DELETE /products/{id} – delete/deactivate a product according to the implementation.

8.4 Order API Test

- POST /orders – create an order with a valid product/customer request.
- Verify the order is stored in RDS MySQL.
- Verify inventory is reduced after a successful order.
- Verify order status/history is recorded according to the application logic.
- GET /orders and GET /orders/{id} – verify retrieval.
- PATCH /orders/{id} – verify permitted status/order updates.

9. Inventory and Alert Validation

- Set a product stock count above the low-stock threshold and verify normal inventory status.
- Reduce stock below the configured threshold but above zero and verify the low-stock metric/alert behavior.
- Set stock to zero and verify out-of-stock metric/alert behavior.
- Check CloudWatch logs/metrics and SNS notifications when alarms enter ALARM state.
- Verify that inventory values shown by the dashboard agree with the application/database state.

10. Report Generation

- EventBridge triggers the report process according to its configured schedule.
- Report Lambda reads the required business/inventory data.
- The generated report is stored in the configured S3 reports location.
- Verify the current report appears in the dashboard.
- Verify the report download function works.

- If a report is missing, check EventBridge invocation, Lambda logs, database connectivity, and S3 permissions.

11. Flask Dashboard Operations

The dashboard is hosted on EC2 and provides a centralized operational view.

- Log in using the configured dashboard/admin token.
- Verify total products and total orders.
- Verify low-stock and out-of-stock information.
- Verify system-health/CloudWatch widgets.
- Verify generated reports are listed.
- Verify report download works.
- For dashboard failures, check EC2 status, application logs, Python dependencies, security-group rules, and AWS permissions.

12. Monitoring and Logs

- Lambda logs: AWS CloudWatch Logs.
- Application metrics: CloudWatch custom/application metrics as configured.
- Inventory alarms: CloudWatch Alarms.
- Notifications: SNS topics/subscriptions configured by the monitoring stack.
- Dashboard: CloudWatch widgets plus application-level business/report information.

13. Troubleshooting Runbook

Problem	Actions
GitHub Actions failed	Open the failed workflow job and inspect the first failing step. Check template validation, OIDC configuration, IAM role trust policy, permissions, parameters, and AWS Region.
OIDC/AssumeRole failure	Confirm the GitHub OIDC provider exists, the IAM trust policy matches the repository/branch, and ARN_ROLE points to the correct role.
CloudFormation stack failed	Open the stack Events tab and identify the first CREATE_FAILED/UPDATE_FAILED resource. Fix the root cause before retrying.
Lambda cannot connect to RDS	Check Lambda VPC/subnets/security group, RDS security-group rules, database availability, credentials/SSM parameters, and Lambda CloudWatch logs.
API returns 401	Check Authorization header, token value, authorizer logs, customer record, and API Gateway authorizer configuration/cache.
API returns 5xx	Check the target Lambda CloudWatch logs, environment variables, permissions, database connectivity, and recent deployment changes.
Inventory metric incorrect	Compare the database stock values with the report/metric Lambda logic and CloudWatch metric timestamps. Confirm the correct metric dimensions are used.
Alarm not triggering	Check alarm threshold, period, evaluation periods, metric namespace/name/dimensions, and whether the metric is actually being published.
SNS notification not received	Check SNS topic, subscription confirmation/status,

	alarm action, and whether the alarm actually entered ALARM state.
Report not generated	Check EventBridge schedule/rule, target Lambda invocation, Lambda logs, DB connectivity, S3 write permissions, and the generated object's timestamp.
Dashboard unavailable	Check EC2 instance status, Flask process/service, port/security-group configuration, Python dependencies, and AWS API permissions.

14. Rollback / Recovery

10. If a deployment fails, stop making unrelated changes and identify the first failing CloudFormation resource.
11. Review CloudFormation stack Events and GitHub Actions logs.
12. If the application worked before the change, revert the problematic Git commit and redeploy.
13. For a stack update, use CloudFormation rollback/retry options as appropriate.
14. Do not manually delete dependent resources just to force a deployment unless the impact is understood.
15. After recovery, rerun the post-deployment validation tests.

15. Operational Checklists

15.1 Deployment Checklist

- ☐ Code committed and pushed to the correct branch
- ☐ GitHub Actions workflow started
- ☐ Template validation passed
- ☐ OIDC authentication succeeded
- ☐ IAM deployment role assumed successfully
- ☐ Network stack succeeded
- ☐ Data stack succeeded
- ☐ IAM stack succeeded
- ☐ Auth stack succeeded
- ☐ Product stack succeeded
- ☐ Order stack succeeded
- ☐ Monitoring stack succeeded
- ☐ Schema initialization succeeded
- ☐ API tests passed
- ☐ Dashboard accessible
- ☐ CloudWatch alarms/dashboard available

15.2 Incident Checklist

- ☐ Identify affected component
- ☐ Check CloudWatch logs/metrics
- ☐ Check CloudFormation stack status
- ☐ Check recent GitHub deployment
- ☐ Check database connectivity
- ☐ Check API authorization
- ☐ Check EventBridge/SNS if reports or alerts are affected
- ☐ Apply the smallest safe fix
- ☐ Validate the application after recovery

- ☐ Document the root cause and resolution

16. Quick Reference

Main repository: GitHub – KundellaAnjali/cloudmart

Primary deployment mechanism: GitHub Actions + GitHub OIDC + CloudFormation/SAM

Backend: API Gateway + Lambda

Database: RDS MySQL

Reports: Report Lambda + S3

Monitoring: CloudWatch + Alarms + SNS

Dashboard: Flask application on EC2

17. Documentation Notes

This runbook is based on the CloudMart project structure and implementation details provided for the project. Before production use, verify exact parameter names, AWS resource names, thresholds, schedules, IAM role ARNs, and secret locations directly in the current CloudFormation templates, workflow, Lambda code, and AWS account.